

markdown

Description of the regression:

One-line description of the regression in behavior.

Bug report: [Issue link]

@author (if internal) @approvers Please could you reply to this comment, copying the questions below and giving some short answers?

1. Were you aware this MR was reverted in the course of your normal work (e.g. through email notification, general work process)?

1. Did you identify the problematic behavior before approving this MR?

1. If not, what would've made the regression more obvious during review?

1. What changes to our tooling or review process would have prevented this regression from being merged?

1. Were the steps to test the MR mentioned clearly in the description? Were they easy to follow?

1. Do you have any other comments/suggestions?

Please reassure the participants that the purpose is not to apportion blame but to gather data, identify causal factors and implement corrective actions - but ask for a swift and brief response while the information is still fresh.

Technical Debt

The ~"technical debt" label, used in combination with ~"devops::plan," helps track opportunities for improving the codebase. These labels should be applied to issues that highlight:

- improvements to existing code or architecture;
- shortcuts taken during development;
- features requiring additional refinement;
- any other items deferred due to the high pace of development.

For example, a follow-up issue to resolve non-UX feedback during code review should have the ~"technical debt" label.

Issues marked with this label are prioritized alongside those proposing new features and will be scheduled during milestone planning.

UX

The Plan UX team supports Product Planning, Project Management and Optimize. Product Planning and Project Management are focused on the work items architecture effort. This page focuses mainly on the specifics of how we support this, since it requires alignment and cross-group collaboration.

UX issue management, weights and capacity planning

Product Planning, Project Management and Optimize will create issues for UX work and pre-pend the title with [UX]. Here is an example - <https://gitlab.com/groups/gitlab-org/-/epics/10224note1337213171>+

- UX issues are the SSOT for design goals, design drafts, design conversation and critique, and the chosen design direction that will be implemented.
- Product requirement discussions should continue to happen in the main Issue or Epic as much as possible.
- When the Product Designer wants to indicate that the design is ready for ~"workflow::planning breakdown", they should apply this label to their issue, notify the PM and EM, and close the issue.

When should a UX issue be used?

UX issues should be used for medium or large projects that will take more than one dev issue to implement (e.g., end-to-end flows, complicated logic, or multiple use cases / states that will be broken down by engineering into several implementation issues). If the work is small enough that implementation can happen in a single issue, then a separate [UX] issue is not needed, and the designer should assign themselves to the issue and use workflow labels to indicate that it's in the design phase.

Weighting UX issues

All issues worked on by a designer should have a UX weight before work is scheduled for a milestone.

- Issue weights should follow the UX Department's definitions.
- If the issue is a dedicated [UX] issue, then the issue weight can be added to the weight field, but it should also be duplicated as a ~'design weight:' label. This is for UX Department planning purposes. For smaller issues where implementation and UX work happen in the same issue, UX weight should be added using the ~'design weight:' label (the weight field is used by engineering).
- Product Managers and Product Designers can use issue weights to ensure the milestone has the right amount of work, to discuss tradeoffs, or to initiate conversations about breaking work into smaller pieces for high-weight items.

Work Items

When designing for objects that use the work items architecture we will follow this process intending to ensure that we are providing value-rich experiences that meet users needs. The work items Architecture enables code efficiency and consistency, and the UX team supports the effort by identifying user needs and the places where those needs converge into similar workflows.

About work items

The first objects built using the work items architecture support the Parker, Delaney and Sasha personas in tasks related to planning and tracking work. Additional objects will be added in the future, supporting a variety of user personas.

Read more about work items

- Work items architecture
- Work items terminology

Terminology

Work items refers to objects that use the work items architecture. You can find more terms defined related to the architecture here: [work items terminology](#).

When we talk about the user experience, we avoid using the term 'work items' for user facing concepts, because it's not specific to the experience and introduces confusion. Instead, we will use descriptors specific to the part of the product we're talking about and that support a similar JTBD. Here are examples of how we are categorizing these:

- Team Planning Objects: Objects that belong to the Planning JTBD. Currently these are Epics, Issues and Tasks but could include others in the future.
- Strategy Objects: Objects that support strategic, organization wide objects. Currently these are Objectives and Key results.
- Development/Build Objects: Objects that support development tasks. These could be MRs, Test Cases, or Requirements
- Protecting Objects: These may include Incidents, Alerts, Vulnerabilities, Service Desk Tickets

This enables us to differentiate these by persona and workflow. While they may share a common architecture on the backend and similar layout on the frontend, in the UI they may:

- appear in different workflows and areas of the application
- have different data fields
- have different actions users can take on them

Guiding principles

- The DRI for the user experience is the Product Designer assigned to the group that is using the work item architecture for their object(s).
- We work in a user-first mindset, rather than technology-first.
- Pajamas is our design system and new patterns introduced via work item efforts need to solve a real problem that users have, be validated by user research, and follow the Pajamas contribution process.
- We follow Pajamas principles for the user experience.
- MVCs provide value to users, are bug-free and a highly usable experience, as described in Product Principles.

How the architecture is intended to work

When designing with the work items architecture, Product Designers should understand roughly how the architecture works and what implications exist for the user experience.

- A work item has a type (epic, incident), and this controls which widgets are available on the work item and what relationships the work item can have to other work items and non-work item

objects.

- The behavior of the work item in terms of performing its targeted JTBD(s) is powered by the collection of widgets enabled for a work item type.
- We want to avoid building logic or views specific to a type. When you need to support a workflow that isn't currently supported, you can introduce new behaviors through widgets (fields, apps, actions). A practical example: Epics can parent other Epics and Issues. Instead of interconnecting epics and issues this behavior is encapsulated in a 'hierarchy' widget, which could be utilized in other work item types that implement hierarchies; such as Objectives and Key Results.
- Similarly, the work item view should not be customized directly for a type. However, the Product Designer can propose a different user experience and the team implementing the work item will incorporate the necessary use cases into the work items architecture.
- Work items can be organized and presented to users in any groupings from an IA/Nav standpoint so long as all views leverage the same SSoT grouping FE components (ex: list, board, roadmap, grid, ...). We should only ever need to build and maintain one version of each grouping view that can then be re-used across anywhere we want to display that set of work items. Groupings are determined iteratively based on user needs.

If the quad discovers that the desired user experience would require a greater contribution to the work item architecture than initially thought, they would discuss trade-offs as a team in order to decide whether to proceed or leave the object separate.

Design Process for Work Items

Problem Validation

The quad that owns the code for the object (incident, epic, etc) decides if something should use the work item architecture based on trade-offs around code reuse and user experience. This should be a cross-functional decision, and the group Product Designer should advise their team regarding how well the user's ideal workflow could or could not be supported by the work items architecture. This will allow the team to evaluate how much existing frontend pieces of the architecture could be re-used, and what would need to be added or customized in order to support the desired experience.

1. As part of the decision making process, Product Designers should do problem validation user research (or leverage existing) to understand the desired user experience, including user goals, tasks, content/data field needs, and whether or not this work item type has relationships and the nature of those relationships.

1. During this phase, the Product Designer and Product Manager should ensure that success metrics are defined per our work item research process (link TBD)

1. High level wireframes should be produced to ensure everyone has a shared understanding of what is wanted and to establish a medium term vision for the work.

Solution Validation

After the quad decides the work item architecture is suitable, the Product Designer will design the experience in detail. As part of the detailed design, Product Designers, in collaboration with the quad, will:

1. Design how existing widgets will be utilized, and any new widgets needed or if existing

widgets could be abstracted to fit a new use case. For example: The Timeline widget for incidents was designed in isolation specific to the incident use case. It could be reworked slightly to support more use cases, such as objective or key result check-ins.

1. Define how users will access this work item. Design how this work item will appear in existing views, such as lists, or any new views needed for this work item.

- Ensure new components and patterns are contributed back to Pajamas.

1. Solution validation should be conducted as needed to ensure the workflow and usability meets the user needs.

Research Process for Work Items

We use the methods and tools in the UX Research handbook.

In addition to these, we're working on gaining an efficiency bonus by using a common screener and building a mini-database of qualified participants aligned to our research needs.

We do a confidence check at different points in the process, particularly before moving a design into the build phase. Sometimes, a design solution is straightforward enough where we're very confident to move ahead without solution validation. However, there are times when we're unsure how the design solution will perform in production, thereby resulting in a low level of confidence. When this happens we will do usability testing to build confidence.

UX Paper Cuts

The UX Paper Cuts team has a dedicated role addressing Paper Cuts concerns within the Plan stage.

The UX Paper Cuts team member covering Plan will regularly triage the list of UX Paper Cuts issues that are related to the Plan stage as outlined above, but will also add actionable candidates to a Plan-specific epic for transparency.

Triaged Plan-specific Paper Cuts issues can be found in <https://gitlab.com/groups/gitlab-org/-/epics/12061>. Currently, quarterly child epics will be created under that parent epic to organize work.

See further details at <https://handbook.gitlab.com/handbook/product/ux/product-designer/suggesting-paper-cuts-to-the-team>

Plan Weekly Digest

Background

In Plan we use async Weekly updates, called Plan Weekly digests, to communicate progress on important work this week to our team members.

The Engineering Managers in the Plan stage alternate each week as the DRIs. There are 3 groups in the Plan stage, and one SEM, so every EM is the DRI roughly once / 4 weeks.

The responsibility of the DRI is simply to ensure the issue is ready to be publicized in time for the coming week by reminding everyone to contribute. All team-members are welcome to

participate in suggesting content using discussions or adding it directly by editing the description.

Process

1. A new confidential issue is created every Monday, 8 UTC. (automatically)
1. The issue is assigned to a Plan Engineering Manager according to the schedule below. Their role is to remind others to contribute.
1. On Saturday, 8 UTC all team members are reminded to read the updates on the issue via a comment (automatically).
1. On Friday, 8 UTC (next week) the issue is closed.

DRIs

Issue creation (auto) DRI
--- ---
2025-04-14 John Hope
2025-04-21 John Hope
2025-04-28 Vladimir Shushlin
2025-05-05 Donald Cook
2025-05-12 John Hope
2025-05-19 Vladimir Shushlin
2025-05-26 Donald Cook
2025-06-02 John Hope
2025-06-09 Vladimir Shushlin
2025-06-16 Donald Cook
2025-06-23 John Hope
2025-06-30 Vladimir Shushlin
2025-07-07 Donald Cook
2025-07-14 John Hope
2025-07-21 Vladimir Shushlin
2025-07-28 Donald Cook
2025-08-04 John Hope
2025-08-11 Vladimir Shushlin
2025-08-18 Donald Cook
2025-08-25 John Hope

Links

- Current Weekly Digests
- Closed Weekly Digests
- Issue Template

Meetings

Most of our group meetings are recorded and publicly available on YouTube in the Plan group playlist.

<iframe width="560" height="315"
src="https://www.youtube.com/embed/videoseries?list=PL05JrBw4t0KoceqcTneOVmAzhEp6NinY0"

frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>

Weekly group meeting

Plan held a weekly team-meeting as a stage until 2023-11-01. The agenda is still available.

The meeting was removed as its functions are now covered in other ways:

- Slack
- Stage Working Groups
- Group meetings
- Smaller ad-hoc meetings
- Social call

Links / References

- ~group::project management
 - Board
 - Roadmap
- ~group::product planning
 - Board
 - Roadmap
- ~group::optimize
 - Board
 - Roadmap
- ~group::knowledge
 - Board
 - Roadmap

Shared calendar

There is a shared Plan stage calendar which is used for visibility into meetings within the stage.

- To add this shared calendar to your Google Calendar do one of the following:

- Visit this link (GitLab internal) from your browser.
- Click the '+' next to 'Other calendars' in Google Calender, select 'Subscribe to calendar',

paste

cb72023177f018d631c852ed1e882e7fa7a0244c861f7e89f960856882d5f549a@group.calendar.google.com
into the form and hit enter.

- To add an event to the shared calendar, create an event on your personal calendar and add Plan Shared as a guest.

Team Day

Team Days are organized on a semi-regular basis. During these events we take time to celebrate wins since the last team day, connect with each other in remote social activities, and have fun!

Anyone can organize a team day. It starts with creating a Team Day planning issue in the plan-

stage tracker and then proceeding to find a suitable date.

Setting a date

A time-boxed vote no more than 3 months but no less than 1 month out has proven to be the most inclusive way to set a date so far. This allows enough time to organize sessions but is usually close enough to avoid colliding with off-sites, or other company-wide activities.

Including at least three major timezones, one for each of AMER, EMEA, and APAC, in the issue description allows people to better see how the day will be divided for them and what they can attend.

It's good practice to rotate the 'base' timezone of the Team Day to spread the opportunity for attendance. For example; the FY23-Q4 Team Day was based on a full UTC day, the FY24-Q3 on a full day AEST.

Sessions

The day is composed of sessions proposed and organized by team-members. These are typically allocated 1hr, though they can be longer or shorter. Sessions can be scheduled in advance to allow other team-members to plan their attendance and participation.

Sessions can be anything really, so long as it aligns with the values. Team-members can organize a game, teach a skill, give a talk on something they know, or anything else they think others might enjoy.

Some examples of sessions we've had on previous team days include:

- A cooking class with a former professional chef.
- Watching a holiday film together.
- Lateral Thinking Games.
- A home woodworking workshop tour and demonstration.
- Remote games; such as Gartic Phone and DrawSaurus.

Free time slots can be used on the day to hold impromptu events requiring little or no preparation.

Participation

Participation in team day is encouraged for any team-member or stable counterpart in Plan. If you collaborate with Plan team-members on a regular basis you're also very welcome to attend.

Participating team-members are encouraged to drop non-essential work and take part in any sessions during the day that they wish to. Those assigned to essential work; such as critical bugs, incidents, or IMOC shifts, are encouraged to participate between their other obligations.

Team day is a normal workday for those choosing not to participate.

Expenses

Some sessions may require small purchases to participate fully; for example, ingredients for a cooking class or hosting of a private video game server.

Unless communicated in advance these are not expensable.

The DRI for organizing Team Day may pursue a budget for expenses under existing budgets; such as the team building budget. If successful it should be made clear to team-members well in advance:

- What purchases qualify for reimbursement.
- The policy the expense qualifies under; including handbook link, policy category, and classification in Navan.
- Any additional handbook guidance that will help team-members utilize the budget.

Past Team Days

- 2023-10-13 Team Day
- 2022-12-20 Holiday Party

Tips for a Successful Team Day

- Watch out for Daylight Savings Time when organizing for Q1 and Q3. When the date is set, check that the timeszones in the planning issue still match the timezones in use on the day (for example, AEST vs. AEDT).
- Secure expense budget and communicate at least a week in advance of the Team Day.
- Ensure Google Calendar events are transferred from the planning issue to the Plan Shared Calendar a week in advance of the event date.
- Ensure everyone has access to the calendar, and have easy step-by-step directions for creating a new event on the calendar (Adding events to a shared calendar can be slightly confusing).
- Communicate this change in SSOT, and encourage participants to add their own sessions in the calendar as free slots.

Team Process

Each group within the Plan stage follows GitLab's product development flow and process. This allows for consistency across the stage, enables us to align with other stages and stable-counterparts, and enables us to clearly understand our throughput and velocity. We're currently focused on strictly following the process stated in the handbook, as opposed to creating our own local optimizations.

In some cases we need to dogfood a new Plan feature that may adjust our adherence to the GitLab's process. If that happens we assign a DRI responsible for setting the objective, reporting on the outcomes and facilitating feedback to ensure we prioritize improvements to our own product. This ensures we're not making a change for the sake of making changes, and gives us clarity into our own evaluation of a change to the product.

In some cases we need to dogfood a new Plan feature that may adjust our adherence to the GitLab's process. If that happens we assign a DRI responsible for setting the objective, reporting on the outcomes and facilitating feedback to ensure we prioritize improvements to our own product. This ensures we're not making a change for the sake of making changes, and gives

us clarity into our own evaluation of a change to the product.

There are a couple of process-related improvements we'll continue to adopt:

- Iterations: We've recently started organizing the prioritized work in a given milestone into weekly iterations. This doesn't change any of the canonical process, and allows us to break a months worth of work into sizeable timeboxes. Intended outcome: Dogfood iterations (the feature), improve velocity and give more granular visibility into the progress of issues. DRI: @donaldcook

Stage Working groups

Like all groups at GitLab, a working group is an arrangement of people from different functions. What makes a working group unique is that it has defined roles and responsibilities, and is tasked with achieving a high-impact business goal fast. A working group disbands when the goal is achieved (defined by exit criteria) so that GitLab doesn't accrue bureaucracy.

Stage Working Groups are focused on initiatives that require collaboration between multiple groups within the stage. The structure of stage working groups is similar to company-wide working groups, with DRI and well-defined roles. The initiatives are driven by a stage-level product direction rather than an Executive Sponsor, and can be formed of just Functional Leads and members who participate in fulfilling the exit criteria.

Active Stage Working Groups

1. Work Items API Performance

Archived Stage Working Groups

1. Epic Work Items Migration

Product Outreach

There can be a gap in understanding between Engineering and Product on a team. We are experimenting with a pilot program that will allow engineers to spend time in the world of Product, with the goal of greater mutual communication, understanding and collaboration. It helps us work more effectively as a team for better features.

Product Shadowing schedule

Engineering team-members can shadow a product stable-counterpart. Shadowing sessions last two working days, or the equivalent split over multiple days to maximize experience with different functions of the role. In particular, the session should include at least one customer call. To shadow a counterpart on the team:

1. Create an issue in the plan project tracker using the Product-Shadowing template;
1. Create a WIP MR to this page to update the table below, adding your name and issue link, and
1. When your counterpart is assigned to the issue, add their name, remove WIP status and assign to your manager for review.

Month	Engineering counterpart	Product counterpart	Issue link
2020-07	Charlie Ablett (@cablett)	Keanon O'Keefe (@kokeefe)	gitlab-org/plan118
2020-10	Jan Provaznik (@jprovaznik)	Gabe Weaver (@gweaver)	gitlab-org/plan185

Speed Runs

- Labels
 - Scoped Labels
- Issues
 - Description Change History
- Epics
 - Organize sub-Epics with the Epic Tree
- Requirements Management
 - Requirements MVC
 - Satisfy Requirements from CI Pipeline

Engineering Scaling Targets

We're tracking a number of issues that we believe could cause scalability problems in the future.

Type	Description	Estimated Timeline for Failure	Resolution Due Date	12 Month Target
Issue	Status			
Redis Primary CPU	Unexpected load on the Shared State Redis instance caused by SUBSCRIBE, UNSUBSCRIBE and PUBLISH commands.	Unknown	November 2023	150k Concurrent WebSocket Connections at peak
Redis Memory	Retention of Action Cable messages in Redis Shared State memory due to high numbers of and/or stalled/hung clients.	Unknown	November 2023	150k Concurrent WebSocket Connections at peak
Primary DB	Scaling a combined 'Work Items' table consisting of all current issues, epics, requirements and test cases.	Unknown	November 2024	50k Work Items created per day

Note: Work is ongoing on migration helpers to mitigate Int4 Primary Key Overflows. These will provide a standard way to resolve these issues.

Current Large Tables

Some tables in our database have grown significantly and may pose scalability issues.

Table Name	Current Size (Bytes)	Planned Mitigation Issue
Status		

```

----- | -----
----- |
| descriptionversions | 424.2 | https://gitlab.com/gitlab-
org/gitlab/-/issues/412704 | <span style='border-radius:0.2em; font-weight:bold; padding-
left:1em; padding-right:1em; color:white; background-color:red;'>Critical</span> |
| issues | 278.4 | https://gitlab.com/groups/gitlab-
org/-/epics/10987 | <span style='border-radius:0.2em; font-weight:bold; padding-left:1em;
padding-right:1em; color:white; background-color:orange;'>Warning</span> |
| resourcelabelevents | 165.3 | https://gitlab.com/gitlab-
org/gitlab/-/issues/412705 | <span style='border-radius:0.2em; font-weight:bold; padding-
left:1em; padding-right:1em; color:white; background-color:orange;'>Warning</span> |
| sentnotifications | 508.1 | https://gitlab.com/gitlab-
org/gitlab/-/issues/417233 | <span style='border-radius:0.2em; font-weight:bold; padding-
left:1em; padding-right:1em; color:white; background-color:red;'>Critical</span> |
| systemnotemetadata | 173.5 |
| <span style='border-radius:0.2em; font-weight:bold; padding-left:1em; padding-right:1em;
color:white; background-color:yellow;'>Warning</span> |

```

By continually monitoring these tables and applying the planned mitigations, we aim to maintain optimal performance and prevent any scalability issues.

Metrics

Plan xMAU

- Product Planning Performance Indicators

SECTION: engineering/devops/dev/plan/internship.md

Overview

The Plan Frontend Team internship is the result of The Engineering Internship Pilot Program that started at the end of 2019. The ultimate goal of this program is to transform an entry-level candidate in to an Individual Contributor who could meet the requirements for a Junior Engineer.

First 30 days

The goals for the internship program are laid out in an epic here:
<https://gitlab.com/groups/gitlab-com/-/epics/614>

Roles

For this program to be successful, the Roles and Responsibilities must be transparent to the Internship Program participants. The primary roles of the program include:

- Frontend Engineering Manager - Manager

- Frontend Engineer - Mentor
- Intern - Participant

Responsibilities

Manager Responsibilities

During the first week:

- Engineering Manager and Mentor Internship Kick Off Meeting
- Create a backlog of small issues and assign to the Mentor, Engineering Manager and the Intern
- Schedule 1:1
- Walk through various team processes
- Set up Coffee Chats for the intern with different team members
- Work with intern to complete Goal Analysis

Daily:

- Review / Respond to the Intern's daily standup
- Serve as resource to the Intern for answering technical or non-technical questions

Weekly:

- Weekly 1-1:
 - Agenda:
 - Discuss Outstanding Questions
 - Review Status of Assigned Issues
 - Review Status of Project
 - 1-1 Questions
 - What went well
 - Learnings from last week
 - Priorities for upcoming week
 - Any challenges or concerns
 - Feedback and career development
 - Assign a new issue from backlog

As needed:

- Add to and reprioritize backlog of small issues
- Provided list of valuable recorded meetings for Intern to watch

Mentor Responsibilities

The Intern will have a Primary Mentor, however any of these responsibilities can be delegated to other team members.

During the first week:

- Engineering Manager and Mentor Internship Kick Off Meeting
- Create a backlog of small issues and assign to the Mentor, Engineering Manager and the Intern

- Schedule welcome meeting
- Schedule weekly synchronous meeting

During the second week:

- Walk through development process from start to finish with the Intern
- Walk through initial issue assigned to the Intern

Daily:

- Check in with the Intern asynchronously
- Serve as a resource to the Intern for answering technical or non-technical questions

Weekly:

- Participate in at least one pair programming session
- Weekly meeting:
 - Agenda:
 - Discuss Outstanding Questions
 - Review status of current issues
 - Review status of Project
 - Discuss learnings from the past week

As Needed:

- Add to and reprioritize backlog of small issues
- Identify best issues to Pair Program with the Intern

Intern Responsibilities

During the first week:

- Complete Onboarding Issue first week items
- Review Plan pages in handbook
 - Frontend Plan Team
 - Plan Stage Page
- Follow the Internship Day to Day Activities
- Review long term Project
- Work with the Engineering Manager to complete Goal Analysis

Weekly:

- Fill out 1:1 agenda with the Engineering Manager prior to 1:1
- Prioritize your work for the week and add to Google Calendar
- Participate in team meetings and activities that fall in your work hours
- Attend Coffee Chats with various GitLab Team Members

Daily:

- Perform daily standup

- Work on MRs / Issues

SECTION: engineering/devops/dev/plan/knowledge.md

Plan:Knowledge team

The Plan:Knowledge team develops Knowledge Management categories:

- Wiki
- GitLab Pages
- Text Editors
- Markdown

Learn more on our direction page.

Team members

{{< team-by-manager-role role="Engineering Manager(.)Plan:Knowledge" >}}

Stable counterparts

{{% engineering/stable-counterparts manager-role="Engineering Manager(.)Plan:Knowledge" role="(.)Plan:Knowledge\$|Product Manager(.)Plan Stage|Security(.)Plan|Engineering(.)Plan\$|Principal(.)Plan\$|Group(.)Plan" %}}

Hiring chart

Check out our jobs page for current openings.

How we work

Picking something to work on

The build board shows upcoming release work. The ~"workflow::ready for development" column is ordered by priority.

Engineering Manager adds the following labels at the start of the milestone:

| Label | Meaning |

| --- | --- |

| ~Deliverable | We have committed to customers that we will deliver this item in the current milestone. |

| ~Stretch | We have not committed to deliver the item but will attempt to make progress on it |

It's OK not to take the top item if you are not confident you can solve it, but please post in gknowledge.

Capacity

Estimating effort

When estimating the effort involved in upcoming work, we use the same approach and numerical scale as other groups in the Plan stage.

```
{{% include "includes/engineering/plan/estimating-effort.md" %}}
```

Typically, 3-month rolling average is a good indicator of the team's capacity. Knowledge is a new team and determining capacity will be difficult at the beginning without clear historical data.

The PM and EM will work to fit ~Deliverable issues into no more than 75% of the team's capacity and allocate the rest to ~Stretch issues.

Refinement

Engineering manager reviews ~"workflow::refinement" issues on every team meeting. Issues with the highest priority are assigned to individual engineers, who are responsible for moving issue to ~workflow::ready for development.

Engineers can put the following template into the issue description:

markdown
Implementation plan

<!--

Ready for development means replying yes to the following questions:

- Is this issue sufficiently small enough? If not, break it into smaller issues
- Is it assigned to the correct domain (e.g. frontend, backend)? If not, break it into two issues for the respective domains
- Is the issue clear and easy to understand? If not, try asking further clarification questions and update the description once they are received

If more than 2 MRs are needed, consider adding a table like the following to the description (e.g. under Implementation plan).

-->

Description MR
- -
MR 1
MR 2
Documentation

Reasoning:

<!--

Add some initial thoughts on how you might break down this issue. A bulleted list is fine.

This will likely require the code changes similar to the following:

- replace the hex driver with a sonic screwdriver
- rewrite backups to magnetic tape
- send up semaphore flags to warn others

Links to previous examples. Discussions on prior art. Notice examples of the simplicity/complexity in the proposed designs.

-->

/label ~"workflow::ready for development"

/label ~"frontend-weight::X"

/label ~"backend-weight::X"

/weight X

Weighing bugs

```
{{% include "includes/engineering/plan/weighing-bugs.md" %}}
```

Refinement

Board-Walk (weekly)

Team-members meet to walk the Build Board once a week. 25 minutes are allocated for this sync call but it may be completed much more quickly than that. The EM is DRI and attendance is optional except for PM. It is recorded and shared in the gknowledge Slack channel. The recording will be made public only if no security or other confidential issues are discussed. The agenda is available internally and all team-members are encouraged to contribute updates and questions.

The purpose of this meeting is to:

- Update on the status of work in progress
- Identify blockers and risk
- Reprioritize
- Ask for help

DRIs should keep issues up to date with workflow labels and health status on an ongoing basis rather than waiting for this meeting.

Planning Meeting (monthly)

A planning meeting is held once per month, prior to the start of the milestone. The Product Manager is the DRI for scheduling it.

Attendance is optional for engineers but participation is not. The meeting will have an agenda

and will be recorded. It may involve any or all of the following:

- Setting priorities and expectations.
- Estimating tasks.
- Breaking down and collaborating on scope.
- Clarifying requirements.
- Estimating capacity and carry-over.

As much as possible these tasks should be completed asynchronously, reducing the work required in the meeting. The purpose of the meeting is to start the upcoming milestone in the best possible shape for success.

Refinement sessions (ad-hoc)

Team-members are encouraged to propose refinement sync meetings for large issues and/or new features.

The goal is to explore concerns and unknowns while sharing knowledge and gathering different perspectives on the problem.

A refinement meeting might have an agenda with topics like:

- Product requirements
- Technical challenges
- Technical alternatives
- How to iterate on the solution proposed

As an outcome, the meeting could produce a list of issues, with an estimated milestone, to iterate over.

Asynchronous-first

Most planning is done asynchronously. Some tools and processes are observed to make this more efficient.

Since issues can only have one milestone attached, the ~"Next Up" label is used to mark items for the upcoming milestone, regardless of whether they already have a milestone or not. PM and EM should remove this label from any issues prior to the start of planning, then add it to prospective issues and any expected to slip the current milestone during the planning process.

Using this label, it's possible to easily analyze the upcoming milestone. The Planning Board mimics the Build Board but is scoped to this label instead of the current milestone. Use it to:

- View the current workflow state of all proposed issues.
- Plan capacity by totalling weight values for each list.
- Understand blocking relationships that may be resolvable before the milestone starts.

When the new milestone starts, the milestone can be added all issues with the ~"Next Up" label in a bulk action, and the label itself removed.

Workflow

Use of Labels

Proper labelling of issues helps with the classification, traceability and quantification of work the team can and is doing. Some labels are essential. The table below describes these and gives the reason why.

Label	Use	Handbook Guidance	DRI
~workflow::	Communicates the current workflow state of an issue. Important for understanding progress & quantifying risk during the course of a milestone.	Updating Issues Throughout Development	Engineer
~type::	Communicates the type of work being done. Used to quantify and report the split of work to roles inside and outside GitLab.	Work Type Classification	
~Deliverable/~Stretch	~Deliverable communicates to customers and stakeholders that we intend to deliver an issue within the assigned milestone. ~Stretch indicates that it might be started during the milestone but is not expected to complete.	Release Scoping Labels	Engineering Manager

Async update

We aim to make the status of each epic and issue clear and easily accessible for our teammates, counterparts and users.

The primary source of truth for this information is the ~workflow:: label and the health status.

But when the issue spends more than a week in the ~"workflow::in dev", ~"workflow::in review" or ~"workflow::verification"

DRI also leaves an async update on the it using the "Knowledge - async update" comment template.

To keep the track of what issues may need an async update, you can use the following GLQL query:

```
markdown
glql
```

```
group = "gitlab-org" and assignee = currentUser() and label in ("workflow::in dev", "workflow::in review", "workflow::verification") and opened = true
```

Priority labels

We use ~Knowledge::P1/P2/P3 labels to indicate issue priority within the ~workflow:: steps and milestones.

- Product Manager is the DRI for these labels, but everybody on the team can assign/adjust them.
- Before the milestone starts, the PM and EM will review the priorities for all issues included

in that milestone. It's expected that:

- 40% of issues have ~Knowledge::P1
- 30% of issues have ~Knowledge::P2
- 30% of issues have ~Knowledge::P3, and those issues can't be ~Deliverable's
- We also use these labels outside of milestones to keep track of our highest priorities.
- If anyone on the team wants an issue to be scheduled, they should add the appropriate priority label.
- We don't have a dedicated P4 label, not having ~Knowledge::P label is equivalent to ~Knowledge::P4.
- When issues are moved to another ~workflow:: stage, it's likely that the priority will be changed.
- ~Knowledge::P labels are completely different from ~priority:: labels that are used only for bugs.

Collaboration with other teams

To avoid rework we reach out to other teams early when working on the following domains:

Team	Domain overlap
Pipeline Authoring	GitLab Pages .gitlab-ci.yml syntax

Dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="knowledge" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}  
  {{< tableau/filters "GROUPLABEL"="knowledge" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}  
  {{< tableau/filters "GROUPNAME"="knowledge" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="knowledge" >}}  
{{< /tableau >}}
```

Detailed metrics are available on the Engineering Metrics page.

Application Performance

Additional dashboards are available in Grafana that show application performance of parts of the application for which the team is responsible.

- Stage-Group Dashboard (including 28-day Error Budget)
- Error Budget Detail

Useful links

- Plan:Knowledge - Apply a milestone filter to see work in the current release
- splan in Slack
- Recorded meetings
- Retrospectives

SECTION: engineering/devops/dev/plan/product-planning.md

Plan:Product Planning team

The Plan:Product Planning team works on both the backend and frontend parts of GitLab's Product Planning category in the Plan stage.

For more details about the vision for this area of the product, see the Plan stage page.

Team members

```
{{< team-by-manager-role role = "Engineering Manager(.)Plan:Product Planning" team = "Product Planning">}}
```

Stable counterparts

```
{{% engineering/stable-counterparts manager-role="Engineering Manager(.)Plan:Product Planning" role="Product Planning|Security(.)Plan|Principal(.)Plan$" %}}
```

Hiring chart

Check out our jobs page for current openings.

Team metrics dashboard

We have a metrics dashboard intended to track against some of the Development Department KPIs, particularly those around merge request creation and acceptance. From that dashboard, the following chart shows MR Rate. Please reference the dashboard section below.

Application performance dashboard

We have useful dashboards tracking the performance of parts of the application we're responsible for:

- Application dashboards; showing request throughput, latency, SQL query counts, cache hits, Sidekiq jobs for Web and API endpoints, git usage and error budgets. Also links to other useful resources in Kibana.
- [Product Planning]
- The Sitespeed Dashboard; showing the results of ongoing synthetic tests against representative product pages. Useful for identifying changes in page load time (TTFB), LCP, etc.

OKRs

Beginning FY24, we have moved away from Ally.io and have been using GitLab to track OKRs. Please note that the links below are only accessible to GitLab team members.

Active Quarter OKRs

The current quarter (FY25-Q3) OKRs are visible here.

Previous Quarter OKRs

All the previous quarter OKRs are available here.

Work

See the Plan stage page and the Plan:Project Management backend team page.

Milestone Planning

The week after a release of current milestone, planning issue for next milestone is created by automation and is available here.

Once the issue is created, Product Manager fills out initial information (i.e. broader theme for the milestone, product priorities, deliverable areas, etc.) within the issue description and refines the Milestone Planning board.

Beginning with 17.8, we have altered the way issues are scheduled for milestones. Refer to following steps on how it is done;

1. Product Manager, in collaboration with Engineering Manager and Product Designer, identifies list of issues that are candidates for upcoming milestone.
 - Number of issues, along with weight and effort is determined based on rolling capacity (i.e. work carried over from previous milestones), available capacity from the team in upcoming milestone, as well as potential time-off and holidays.
 - These issues have ~workflow::planning breakdown label present along with the assigned milestone, implying that they're ready to be triaged by engineering.
2. Identified candidate issues list is posted in the Milestone Planning issue by PM by last week of the calendar month.
3. Engineering Manager then assigns these issues to all the team members two weeks before the start of upcoming milestone.
 - Assignees of issues are picked based on team member availability as well as expressed interest by a team member to work on a specific product area.
 - Additionally, if the assignee is known to be not familiar with the product area of an

issue, SME can be optionally included in the triage notification to encourage collaboration during triage.

4. Along with assignment of issues, EM also includes a comment to notify assignees (and SMEs, if included) that they allocate some time during the week to triage the issues by following Definition of Ready, and wrap it up before the last week of current milestone.

- The goal of this triage exercise is to ensure that assignees know what they'll be working on in the next milestone, ensuring they have all the information needed to begin implementation, and eventually move the issue to ~workflow::ready for development state.

- There can be multiple outcomes during triaging of the issue as follows;

- Issue is too big as a single issue (eg; Weight 4 or higher), in this case, we break down that issue into either sub-tasks (if shipping in the same milestone) or multiple issues (if shipping in multiple milestones).

- Issue has a blocking dependency that either doesn't have a tracking issue or the issue is present in the backlog or future milestone. In this case, we ensure that the relevant issue is brought up in the upcoming milestone to unblock the dependency and is triaged similarly and the original issue is moved out of the milestone in coordination with the Product Manager as per the schedule of blocking issue.

- Issue is not feasible to be implemented at this point, in this case, raise it mentioning PM and EM in the issue so that appropriate steps can be taken.

5. At the end of triaging an issue, if it was determined that issue is ready to be worked on, ensure that issue has following attributes present;

- Workflow label changed to ~workflow::ready for development.

- An implementation plan present within the issue description (this may include technical details too).

- Child tasks or related issues in case original issue was broken down.

- Appropriate weight representing approximate effort.

6. In a week prior to the start of the upcoming milestone, EM's responsibility is to ensure that all candidate issues have either moved to ~workflow::ready for development as per guidance in point 5, or an appropriate triage action is taken as outlined in point 4.

Capacity Planning

Estimating effort

When estimating the effort involved in upcoming work, we use the same approach and numerical scale as other groups in the Plan stage.

```
{{% include "includes/engineering/plan/estimating-effort.md" %}}
```

Weighing bugs

```
{{% include "includes/engineering/plan/weighing-bugs.md" %}}
```

Consider a Spike

Work that arrives in ~"workflow::ready for development" that is out of scope or ill-defined should be

returned to ~"workflow::planning breakdown" for further refinement.

To avoid the disruption this introduces we try to reduce the number of times it happens by planning more carefully. While it's not always possible, we aim

to identify complexity before the build phase, such as by assigning an engineering DRI during the design and validation phase.

However, sometimes complexity can't be accurately estimated until development work starts. If you anticipate this during planning, consider creating a spike and produce a spike report.

Notify the participants in the issue, especially the PM, that a spike is required, create a separate issue and follow these steps:

1. Title the issue with the goal of the spike;
1. Add the ~spike, ~backend, and corresponding stage/group labels;
1. List the unknowns and questions to be answered;
1. List decisions, architectural or otherwise, that need to be made;
1. Identify specializations required to complete the spike (e.g. Backend, Frontend and UX) assign a DRI from each;
1. Mark the issue as blocking the original, and
1. Label with ~"workflow::ready for development" and assign to the current milestone.

The deliverable is a spike report that answers the questions set out in the issue description. This report is typically documented directly in the spike issue, containing the investigation findings, answers to the questions, and links to any PoC MRs produced.

For larger initiatives consider following the recommendations about creating design documents.

Collaborating to Improve Velocity

As a team we often work on features that require close collaboration. We've identified a list of techniques and characteristics that help projects like this proceed at a pace that is sustainable, predictable, and challenging, yet rewarding. An example of such feature was Epic Linking.

1. Feature is designed and broken down in advance of milestone start, including a spike if appropriate.
1. Participants in the spike take part in delivery of the feature.
1. Prior to closing the description is updated with Acceptance Criteria, with sign-off by each assignee + PM. This is what will be delivered.
1. For efforts that are part of larger initiatives (like Work Items), architectural documents are kept up to date with larger decisions; for example, around API design or functionality.
1. Requirements are well-defined with a goal that is achievable within a single milestone and provides business value. For larger features, work may be spread out over several milestones.
1. Items that must be delivered in separate milestones are identified and prioritized first; such as migrations, security issues, and other multi-version compatibility issues.
1. The stable counterpart from documentation is included at the start of the spike.
1. Reviews are kept inside the team where possible to ensure domain expertise, capacity and a low level of context switching.
1. EM and PM work to remove or limit unnecessary/distracting work.
1. Feature is set as primary deliverable.
1. Due dates defined well in advance.
1. DRIs assigned to every upcoming and in-progress piece of work.

1. Use of Slack and synchronous communication to capture regular updates.

1. If team members have PTO during the milestone, make a plan to hand over work in progress to another team member.

Collaborating across disciplines

Workflow Labels

Most issues, especially features, involve working with other disciplines. A single issue will often be shared between frontend and backend and it can be difficult to know which workflow label should be applied, especially when progress is at different stages.

To ensure visibility for other team-members, for issues with a frontend and backend component:

- Assign yourself as early as possible if you have capacity to work on it;
- When both frontend and backend DRIs are assigned, consider hosting a small kickoff discussion.
- When the backend work is merged and verified add the ~"backend complete" label.

We value velocity over predictability

so use your own judgement on whether you should wait for your counterpart to get involved before proceeding with development.

Using the ~"backend complete" label

The ~"backend complete" label is added to issues with multiple specializations (usually backend and frontend) to indicate that the backend component is complete. Backend engineers should add this label when the backend work is functionally complete, merged and verified but frontend, or other work is ongoing.

Definition of Ready

During the planning and breakdown process we usually need to make issues and tasks that outline the technical work required.

A Definition of Ready is an agreed set of guidelines for ensuring an issue or task is ready to be picked up and worked on. Its job is to provide a reference that we can use when deciding whether we can add the 'ready for development' workflow label and expect somebody to be able to complete the work.

To determine whether an issue or task is suitable to be deemed ready for development, consider the following Definition of Ready.

1. The work is described in sufficient detail. A reader without previous context or assumed knowledge is able to understand what work is to be undertaken. What we're doing and why we're doing it is described.
2. Acceptance Criteria or Requirements are clearly defined. This should help the assignee

understand the scope of the work and verify the intended happy/sad paths are functional and tested.

- Including the relevant license level and feature flags in the Acceptance Criteria or Requirements is also advised.

3. Other work that blocks or is blocked by the work is linked with the correct relationship.

4. Relevant resources such as designs, documents, and external links are attached.

5. Cross-functional dependencies are described; such as needing to ask for Technical Writing assistance.

6. Documentation that needs to be updated or added is linked.

7. In case of feature development, a weight has been added and confirmed with at least one other engineer (preferably a domain expert).

- In case an item is weighed as 4 or above, domain expert can provide guidance on breaking down this issue into separate issues (or tasks).

> During the process of populating an Issue or Task for engineering work, we will probably need to ask questions for extra clarity or to help prevent us from making assumptions. We may also feel we don't have enough information to be able to properly break down work into suitably sized chunks. Consider a spike where the path forward is not clear.

Why?

During the planning/breakdown process, it's important that we keep in mind a couple of things:

1. The author of the issue/task may not be the person to work on it
2. We might not start working on the issue/task right away

Our Definition of Ready should help ensure that our work is defined well enough that the two points above are not problems.

Due to our async nature and distributed team, treating a Definition of Ready as a hard quality gate could have a negative effect on productivity. Because of this, the Definition of Ready should be used as a guide rather than as rules and we shouldn't prevent work from being picked up if an issue/task doesn't adhere to the Definition of Ready perfectly.

Issues and tasks are mutable · we can always go back and change plans, add missing context, document decisions made during development, etc.

Health Status

To help with visibility of whether or not an issue can be delivered in a milestone, the team uses health status to communicate quickly.

At the beginning of a milestone, all issues relating to the primary deliverable are assigned the "On track" status. Throughout the milestone, feature DRIs are responsible for updating issues statuses when necessary. If the health status is set to a status that is not "On Track", feature DRIs should leave a comment with additional details about why and if there's a way others on the team can help.

Using health status allows stakeholders, such as product and engineering managers, designers, and other engineers, to get a quick and easy overview of what is unlikely to make it into the current milestone.

Documentation

Documentation should accompany code for any new or changed functionality as per our definition of done. This can become tricky when collaborating on a feature that is behind a feature flag.

Since all feature flags start as disabled by default, we should aim to document the feature as soon as it's safe for testing by users using the feature flag template. Don't wait until a feature is performant and stable to document it, instead do so once it's secure and won't leave data in a corrupt, interim state.

Try to include docs with the first MR to introduce usable functionality. If this is an API addition with no UI, document that and allow the FE engineers to update it as work proceeds. As the feature flag rollout proceeds, the documentation should be updated.

This avoids the rush to provide documentation that often accompanies the release cutoff.

Dashboards

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="product planning" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="product planning" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="product planning" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="product planning" >}}
{{< /tableau >}}
```

Detailed metrics are available on the Engineering Metrics page.

Product Planning is part of a test of new MR sub-type labels which are designed to make it easier to understand which top-level type should be applied. You can read more about them in the Work Type Classification section of the metrics page.

Note: MR Type may differ from issue type. For example, a ~"maintenance::dependency" change that supports a new ~"feature::enhancement".

Picking something to work on

The team Build Board always shows work in the current release, with workflow columns relevant to implementation. Filtering it by ~backend shows issues for backend engineers to work on.

It's OK to not take the top item if you are not confident you can solve it, but please post in splan if that's the case, as this probably means the issue should be better specified.

Direction Items

Items that are customer-facing deliverables and high impact are labeled with ~"direction". We strive to have these items in production by two days before the release cut-off to give ample time for validation.

High Severity Issues

```
{{% include "includes/engineering/plan/high-severity-items.md" %}}
```

Working on unscheduled issues

Everyone at GitLab has the freedom to manage their work as they see fit, because we measure impact, not activity. Part of this is the opportunity to work on items that aren't scheduled as part of the regular monthly release. This is mostly a reiteration of items elsewhere in the handbook, and it is here to make those explicit:

1. We expect people to be managers of one, and we use GitLab ourselves. If you see something that you think is important, you can request for it to be scheduled, or you can work on a proposal yourself, as long as you keep your other tasks in mind.
1. From time to time, there are events that GitLab team-members can participate in, like the issue bash. Anyone is welcome to participate in these.
1. If you feel like you want to have some specific time set aside, but aren't interested in the topics of an existing event, feel free to label issues with "For Scheduling" and copy your manager for visibility.

When you pick something to work on, please:

1. Follow the standard workflow and assign it to yourself.
1. Share it in splan - if not even more widely (like in development or backend).

Useful links

- :Plan:Product Planning - Apply a milestone filter to see work in the current release
- splan in Slack

- Recorded meetings
- Retrospectives

SECTION: engineering/devops/dev/plan/project-management.md

Plan:Project Management Team

The Plan:Project Management team works on
GitLab's Project Management category in the Plan stage.

For more details about the vision for this area of the product, see the
Plan stage page.

Team members

{{% team-by-manager-role "Engineering Manager(.)Plan:Project Management" %}}

Stable counterparts

{{% engineering/stable-counterparts manager-role="Engineering Manager(.)Plan:Project
Management" role="(.)Plan:Project Management\$|Product Manager(.)Plan
Stage|Security(.)Plan|Engineering(.)Plan\$|Principal(.)Plan\$|Group(.)Plan" %}}

Hiring chart

Check out our jobs page for current openings.

Scalability Targets

We're tracking a number of issues that we believe could cause scalability problems in the
future.

Type	Description	Estimated Timeline for Failure	Resolution Due Date	12 Month Target
Issue	Status			
---	---	---	---	---

Primary key int4 overflow	systemnotemetadata.id column is at 50% saturation and must be converted to bigint (int8).	March 2024 - 2025	2023-12-22	Sub-50% 424114 Urgent
Redis Primary CPU	Unexpected load on the Shared State Redis instance caused by SUBSCRIBE, UNSUBSCRIBE and PUBLISH commands.	Unknown	November 2023	150k Concurrent WebSocket Connections at peak Okay
Redis Memory	Retention of Action Cable messages in Redis Shared State memory due to high numbers of and/or stalled/hung clients.	Unknown	November 2023	150k Concurrent WebSocket Connections at peak 326364 Okay

| Various | Scaling a combined 'Work Items' table consisting of all current issues, epics, requirements and test cases. | Unknown | November 2023 | 100k Work Items created per day| |
Okay |

Note: Work is ongoing on migration helpers to mitigate Int4 Primary Key Overflows. These will provide a standard way to resolve these issues.

Work

You can see how we work as a stage at the Plan stage page.

For the backend team specifically, we use the standard GitLab engineering workflow. To get in touch with the Plan:Project Management backend team, it's best to create an issue in the relevant project (typically GitLab) and add the ~"group::project management" label, along with any other appropriate labels. Then, feel free to ping the relevant Product Manager and/or Engineering Manager as listed above.

For more urgent items, feel free to use splan on Slack.

Capacity planning

Estimating effort

When estimating the effort involved in upcoming work, we use the same approach and numerical scale as other groups in the Plan stage.

{{% include "includes/engineering/plan/estimating-effort.md" %}}

Weighing bugs

{{% include "includes/engineering/plan/weighing-bugs.md" %}}

Refining and organizing feature work

To help drive alignment with our stable counterparts, provide visibility into progress, and breakdown our vision into a series of MVCs, we collaborate with Product and UX during ~workflow::planning breakdown to refine and organize ~type::feature deliverables into the following structure:

- Feature (Epic) - Contains all of the necessary vertical feature slices to default the corresponding feature flag to "on". The feature epic will also serve as the location to generate a corresponding Release Post item MR. The feature epic should be scoped to the minimal amount of functionality that still provides customer value. Additional scope planned for future enhancements should be stored in follow-on epics.
- Spike (Issue) - If we are unable to accurately estimate the effort necessary to implement the feature, we first conduct a spike
- UX (Issue) - For larger initiatives, UX creates a separate UX issue that serves as the SSOT for design goals, design drafts, design conversation and critique, and the chosen design

direction that will be implemented. Learn more about UX issues.

- Vertical Feature Slice (Issue) - A subset of the feature that can be completed within a single milestone, tested, and verified within the plan-stage group on production.

- Engineering Tasks (Task - Optional) - One or more engineering tasks that need to be completed in order to deliver the vertical feature slice. The scope of a task should generally correlate to a single MR.

During the ~workflow::planning breakdown phase, all issues need to be weighted so we can efficiently and effectively collaborate with Product and UX on "right sizing" the feature epic. It's advisable that all issues are connected to a parent epic that describes the broader set of improvements we are proposing within a specific area of the product. The desired outcome is to ensure it's as small as possible, maximizes our ability to iterate, and makes it easy to track overall progress on delivery, while providing meaningful value and avoiding an undue amount of "change fatigue" for our customers.

Spikes

There is a decent amount of complexity in the features that we as a team are responsible for, and we're aiming to determine where the majority o